

S-स्पर्श SPARSH

TECHNICAL DESIGN DOCUMENTATION

AI-powered Indian Sign Language Learning & Communication Platform

Team Name: Charlie's Engineers

Member 1: Anagha Kadam

Member 2: Shalmalee Deshmukh

Member 3: Lavanya Nimbalkar

College: Bharati Vidyapeeth College of Engineering for Women, Pune

Live application: <https://isl-translator-final.onrender.com/>

Document Control

Field	Details
Project	SPARSH / S-स्पर्श — Indian Sign Language Translator
Version	1.0
Status	Prototype design; Conversation Mode robustness and accuracy tuning remain ongoing
Deployment	Render; React frontend and FastAPI backend served from one origin
Primary users	ISL learners, signers, families, educators, and people seeking communication support

Contents

- 1. Overview and goals
- 2. System architecture
- 3. Technology stack
- 4. Recognition pipeline
- 5. Static and dynamic model design
- 6. Conversation and translation
- 7. Application modes
- 8. Backend and persistence
- 9. Privacy, performance, reliability
- 10. Deployment and release checklist
- 11. Testing and evaluation
- 12. Limitations and future work
- 13. Team responsibilities
- Appendices

1. Overview and Goals

SPARSH is a web-based Indian Sign Language (ISL) learning and communication platform. It combines guided learning and camera-based practice with Conversation Mode, which turns recognized sign input into readable messages and Marathi output. The objective is a useful workflow beyond isolated sign classification.

1.1 Core objectives

- Offer structured ISL learning with visual sign references and progression-based interactions.
- Give learners live camera feedback and confidence/confirmation cues during practice.
- Recognize a growing vocabulary of static and selected motion-based signs.
- Build words and sentences from confirmed signs and pauses, then provide English and Marathi output.
- Keep hand-landmark detection in the browser where possible, avoiding routine transmission of raw webcam frames.
- Keep the architecture modular so models and learning content can evolve independently.

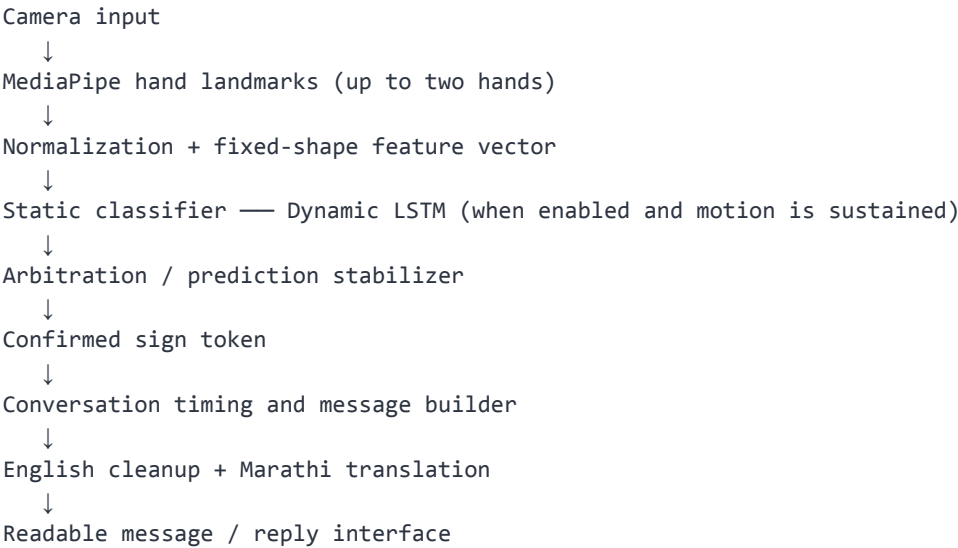
POSITIONING SPARSH is an assistive learning and communication prototype, not a guarantee of perfect recognition, a complete translator for every ISL sign, or a replacement for professional interpreters.

2. System Architecture

The system separates high-frequency camera processing from UI rendering and separates sign recognition from sentence formation and language cleanup.

Layer	Technology	Responsibility
User interface	React + Vite	Learning screens, camera views, recognition feedback, progress, message building, chat.
Camera scheduling	Browser camera APIs	Capture permission and frame sampling.
Landmark inference	MediaPipe Tasks Vision in Web Worker	Detect up to two hands locally without blocking the main UI thread.
Feature preparation	JavaScript normalization	Prepare fixed-shape landmark features and masks.
Static recognition	Static classifier exported to ONNX	Classify held hand configurations.
Dynamic recognition	PyTorch LSTM + ONNX Runtime Web	Classify selected motion signs from a sequence when dynamic assist is active.
Stabilization	Frontend arbitration logic	Use movement, confidence, repeated predictions, and cooldown.
Application API	FastAPI	Translation, authentication, compatibility classification, dataset-related endpoints, health checks.
Language output	Groq cleanup helper + Marathi translation	Convert completed ISL-style gloss to readable English and Marathi.
Hosting	Docker + Render	Build and serve frontend/API from the same origin.

2.1 Logical data flow



DESIGN SEPARATION Recognition estimates WHAT sign was shown. The conversation timing layer decides WHEN a confirmed token becomes part of a word or sentence.

3. Technology Stack

Technology	Role	Reason
React	Frontend UI	Reusable screens and interactive modes.
Vite	Build tooling	Produces production frontend assets.
MediaPipe Tasks Vision	Hand landmark detection	Converts images into structured hand coordinates.
Web Worker	Background browser processing	Avoids blocking the UI with repeated inference.
Scikit-learn classifier	Static recognition	Classifies a prepared feature vector.
PyTorch LSTM	Dynamic recognition	Learns temporal patterns across ordered frames.
ONNX / ONNX Runtime Web	Model deployment	Allows supported trained models to run in a browser.
FastAPI + Uvicorn	Backend	Application endpoints and frontend static serving.
Groq helper	Optional language cleanup	Processes completed gloss, not every frame.
Marathi translation layer	Bilingual output	Produces Marathi text for completed messages.

SQLite	Demo account persistence	Simple prototype storage; production persistence needs review.
Docker + Render	Deployment	Packages the app and hosts it under one URL.

4. Recognition Pipeline

4.1 Camera capture and scheduling

The browser requests camera permission and samples frames using `requestVideoFrameCallback` where available, with `requestAnimationFrame` as a fallback. Inference resolution can be independent of display resolution to control computational cost.

4.2 Hand landmarks

MediaPipe detects up to two hands and returns 21 landmarks per hand. Each landmark contains x, y, and z values; one hand therefore contributes 63 values. Handedness, drawing coordinates, and masks for missing hands are managed separately where required.

4.3 Normalization and fixed shape

Landmarks are normalized to provide a consistent model input and reduce dependence on raw pixel position. The documented browser feature representation has 128 values. Fixed-shape features and masks support one-hand, two-hand, or temporarily missing-hand cases.

4.4 Inference and stabilization

- Static recognition is the default for held hand configurations.
- Dynamic recognition is for selected motion signs and is used when dynamic assist is active.
- Predictions are not committed blindly on every frame; confidence, repeated labels, movement persistence, and cooldown help stabilize output.
- The worker sends compact landmarks and prediction results to the UI rather than passing raw camera frames through React state.

5. Static and Dynamic Model Design

5.1 Static classifier

The static classifier evaluates a prepared landmark vector for signs represented by a held pose. Project documentation describes a Scikit-learn model exported for browser use through ONNX. The documented static labels are A, B, C, D, E, H, I, K, M, S, SPARSH, T, U, and V. Verify labels against the model metadata bundled with the exact release.

5.2 Dynamic LSTM

Dynamic signs depend on how hands move over time. A single frame may not contain enough information, so SPARSH uses a Long Short-Term Memory (LSTM) model for selected motion-based signs.

Model detail	Documented configuration
Input	30 frames × 128 features
Architecture	Two-layer LSTM, hidden size 128, dropout, linear classification head
Training framework	PyTorch
Browser deployment	ONNX + ONNX Runtime Web
Recorded dataset	495 sequences across 14 labels
Validation result	96.97% held-out sequence validation accuracy reported in project documentation; not a real-world guarantee

Label metadata	Documentation contains an inconsistent label listing; verify exact names against the deployed model label file.
----------------	---

Do not present validation accuracy as an independent-user or production benchmark. Confirm the label count and names from the model metadata before publication.

5.3 Static/dynamic arbitration

The arbitration layer acts as a referee between recognition paths. Static recognition remains the default; a weak dynamic prediction should not override a strong static prediction. Dynamic activation should require sustained movement, sufficient confidence, and temporal consistency.

1. STATIC: evaluate held poses and suppress tiny movement jitter.
2. MOVEMENT CANDIDATE: detect sustained movement above the entry threshold.
3. DYNAMIC LOCK: allow sequence predictions while movement remains meaningful.
4. DYNAMIC COMMIT: accept a sign only after confidence and stability checks pass.
5. COOLDOWN: briefly prevent immediate re-triggering after a dynamic commit.
6. RETURN TO STATIC: resume static recognition when movement falls below the exit threshold.

THRESHOLDS Use separate enter/exit thresholds and a short cooldown, calibrated against the current normalization and tested on real recordings. Do not treat suggested values as universal constants.

6. Conversation and Translation

6.1 Message formation

After recognition is stabilized, the conversation engine builds a message from confirmed tokens. The documented Conversation Mode uses approximately 1.5 seconds of pause as a word boundary and approximately 4 seconds as a sentence boundary. These settings may need tuning for different signing speeds and pauses.

6.2 Language processing

- Confirmed signs/gloss are accumulated before cleanup.
- An optional Groq helper converts completed gloss into more readable English.
- The Marathi translation layer produces Marathi output for the completed message.
- If cleanup is unavailable, a basic capitalization/punctuation fallback keeps the flow usable.
- LLM calls are excluded from frame-by-frame inference to reduce latency, cost, and the risk of rewriting uncertain predictions as confident sentences.

6.3 Two-way interaction

The interface can display a typed response from a non-signer. Describe any text-to-speech capability only according to the exact deployed implementation; a response UI alone does not prove that speech synthesis is active.

7. Application Modes

Mode	User flow	Key behavior
Learning	Choose lesson → view sign reference → progress	Guided journey and progression-based UI.
Practice	Start camera → show sign → receive feedback → hold to confirm	Documentation describes a two-second steady hold; SPACE and FULLSTOP controls/signs are available in this mode.
Compete	Receive target → practise → confirm → next checkpoint	Challenge-style practice with hold confirmation.

Conversation	Enable camera → sign → stabilize tokens → build message → bilingual output	Selected dynamic signs are supported; robustness tuning remains ongoing.
Account	Sign up/sign in → use profile features	SQLite account persistence documented for the demo.

Conversation Mode relies on pauses to form words and sentences. Practice Mode can use explicit SPACE and FULLSTOP controls. Do not assume identical timing or punctuation behavior across modes.

8. Backend and Persistence

Area	Current design	Considerations
Frontend	Built React/Vite assets served by FastAPI	Single origin simplifies API requests.
API	FastAPI / Uvicorn	Keep high-frequency browser inference separate from slower language/API requests.
Authentication	Demo sign-up/sign-in flow	Review password handling, session lifetime, and account lifecycle before production.
Database	SQLite for demo accounts	Move to managed/persistent storage for durable multi-user production use.
Model artifacts	ONNX files in frontend public models	Version model and label metadata together; check browser cache after updates.
Language service	Groq key configured in Render environment	Never commit API keys; retain fallback behavior.

The main hand-landmark detection pipeline runs locally in the browser; raw camera frames are not sent to the backend for MediaPipe detection. This reduces routine video exposure but is not a blanket privacy certification: authentication, translation inputs, and explicitly submitted recordings should be reviewed separately.

9. Privacy, Performance, and Reliability

Decision	Purpose	Monitor
MediaPipe in Web Worker	Keep repeated inference off the UI thread	Worker startup failures or crashes.
Raw frames remain local for landmark detection	Reduce video exposure and network dependency	Ensure future changes do not introduce unexpected uploads.
Fixed-size features and masks	Consistent model input shape	Missing-hand masks and normalization mismatch.
Reuse model initialization	Avoid repeated model loads	Stale/cached model assets after deployment.
Temporal buffers outside React state	Avoid unnecessary re-renders	Incorrect buffer reset or stale sequence data.
Confidence, consensus, movement gating	Reduce false dynamic commits	False positives during idle/transition motion.
Cleanup after sentence completion	Reduce latency and cost	Translation provider failures.

9.1 Graceful degradation

- If camera permission is denied, explain the permission requirement and keep non-camera learning content available where possible.
- If the dynamic model is unavailable, show its state clearly and do not imply dynamic recognition is active.
- If language cleanup fails, use the documented fallback and avoid implying the result received full cleanup.
- If translation fails, preserve the recognized source message rather than silently discarding it.

10. Deployment and Release Checklist

The single Render service uses Docker to build the Vite frontend, install the Python runtime, copy frontend assets into the FastAPI service, and start Uvicorn. FastAPI serves the frontend and API from the same origin.

Component	Role
Dockerfile	Builds frontend with Node, installs Python dependencies, packages the app.
Render	Hosts https://isl-translator-final.onrender.com/ .
FastAPI static serving	Serves frontend assets and API routes from one origin.
Environment variables	Provide secrets/configuration such as the Groq API key.
ONNX artifacts	Included in frontend public models and loaded by the browser worker.

10.1 Release checklist

- Confirm model artifacts and label metadata are present in the build.
- Confirm dynamic model status in the UI matches deployed artifacts; do not demo with a stale 'not exported' status.
- Test camera permission, front/back camera behavior, and mobile layout.
- Test static recognition, dynamic assist, idle false activations, message order, boundaries, and translation fallback.
- Check that secrets, local environment files, virtual environments, and unintended local database files are not committed.
- Test the live Render URL after deployment; hard-refresh after model updates to avoid stale cached assets.

11. Testing and Evaluation

Evaluate offline model performance separately from end-to-end conversation quality. Sequence validation accuracy alone does not measure false activations, sentence quality, latency, or performance for unseen signers.

Test area	Measurements	Evidence
Static classifier	Per-label precision/recall, confusion matrix, confidence calibration	Held-out examples and documented confusion pairs.
Dynamic classifier	Per-label metrics, sequence accuracy, false activations per minute	Held-out signers and idle/transition sequences.
Arbitration	False dynamic overrides, missed dynamic signs, commit stability	Recorded sessions with static poses, transitions, and motion signs.
Conversation engine	Token order and word/sentence boundary accuracy	Scripted sequences with expected output.
End-to-end UX	Feedback latency, inference latency, UI responsiveness	Desktop and mobile test results.
Language output	Meaning preservation, English readability, Marathi quality, fallback	Human-reviewed examples including provider failure.
Robustness	Signer, lighting, background, orientation, camera variation	Test set separated by signer and capture conditions.

11.1 Dataset split

Keep training, validation, and test recordings separated by signer where feasible. A held-out signer set is more informative about generalization than randomly splitting near-duplicate clips from the same person. Record capture conditions and balance samples across labels.

12. Limitations, Risks, and Future Work

Limitation / risk	Why it matters	Recommended next step
Limited recording diversity	Performance may fall for unseen users and environments.	Collect balanced recordings across signers and conditions.

Conversation robustness	Transitions, static poses, motion signs, and pauses must be separated.	Add idle/transition samples and tune thresholds on recorded sessions.
Selected dynamic vocabulary	The LSTM does not cover all ISL vocabulary or grammar.	Expand vocabulary incrementally and evaluate every label.
Mobile back-camera coverage	Current data may not represent back-camera behavior.	Collect dedicated front/back camera test data.
Validation versus field performance	Sequence score may overstate practical accuracy.	Publish per-label and end-to-end metrics.
SQLite demo persistence	May not meet production durability or concurrency needs.	Adopt managed persistent storage.
Mobile-aware web app	Not equivalent to a native offline mobile application.	Plan native packaging after browser reliability improves.
Sensitive communication context	Incorrect output can misrepresent intended meaning.	Show confidence/confirmation and position the product as assistive.

12.1 Suggested priorities

7. P1 — Recognition reliability: diverse data, held-out signer evaluation, idle/transition examples, per-label metrics.
8. P2 — Conversation quality: reduce false commits, tune boundaries, test sentence order end to end.
9. P3 — Performance: profile inference, consider ONNX optimization/quantization, monitor mobile battery and responsiveness.
10. P4 — Learning content: expand lessons, revision, goals, quizzes, and accessible feedback.
11. P5 — Production readiness: managed persistence, secure account lifecycle, monitoring, model versioning, rollback.
12. P6 — Mobile expansion: dedicated camera testing and offline model packaging.

13. Team Responsibilities and Handoff

Owner	Primary responsibility
Lavanya Nimbalkar	AI/inference: MediaPipe browser processing, normalization, temporal buffer, static/dynamic integration, LSTM, arbitration, and conversation timing.
Shalmalee Deshmukh	Frontend UI: React/Vite screens, responsive/mobile experience, conversation UI, chat, Learning Mode interactions, visual polish.
Anagha Kadam	Backend and integration: authentication, profiles/progress persistence, FastAPI endpoints, database integration, and end-to-end testing.

13.1 Collaboration checklist

- Work on assigned branches and use pull requests; keep main as the shared stable branch.
- Keep static and dynamic datasets organized separately by label; do not overwrite another contributor's recordings.
- After recording data, train the relevant model, export ONNX, verify labels, rebuild frontend, and commit matching artifacts/metadata.
- Before release, test Practice Mode, Conversation Mode static/dynamic paths, message order, camera permissions, mobile camera behavior, and translation fallback.

Appendix A — Key Terms

Term	Meaning
ISL	Indian Sign Language.
Landmark	A detected reference point on a hand, represented by coordinates.
Normalization	Transforming coordinates into a consistent representation for the model.

Static classifier	Model that classifies a held pose from a prepared feature vector.
LSTM	Long Short-Term Memory neural network designed to process ordered sequences.
ONNX	Interoperable format for representing trained machine-learning models.
ONNX Runtime Web	Runtime that executes supported ONNX models in a browser.
Arbitration	Rules deciding whether a static or dynamic prediction is trusted and committed.
Gloss	A simplified sequence of sign labels used before readable sentence cleanup.
Web Worker	Browser thread that performs work without blocking the main interface.

Appendix B — Project Reference

Live application: <https://isl-translator-final.onrender.com/>

This document reflects the project documentation supplied by the team. Verify implementation details that may vary by branch or deployment—especially label metadata, dynamic model availability, exact movement thresholds, and optional speech behavior—against the code and live build before formal release.

Technical Flow:

